

Undergraduate Lab Report of Jinan University

Course Title Experiment of Computer Organization Evaluation

Lab Name Manipulating Bits Instructor SUN Heng

Lab Address N116, Panyu Campus

Student Name, No.

李荣耀(2024103975)、陈抟飞(2024101329)、杨启承(2024101322)

College International School, Jinan University

Department Major Computer Science & Tecnology

Date 2025 / 10 / 22 Afternoon

1. Introduction

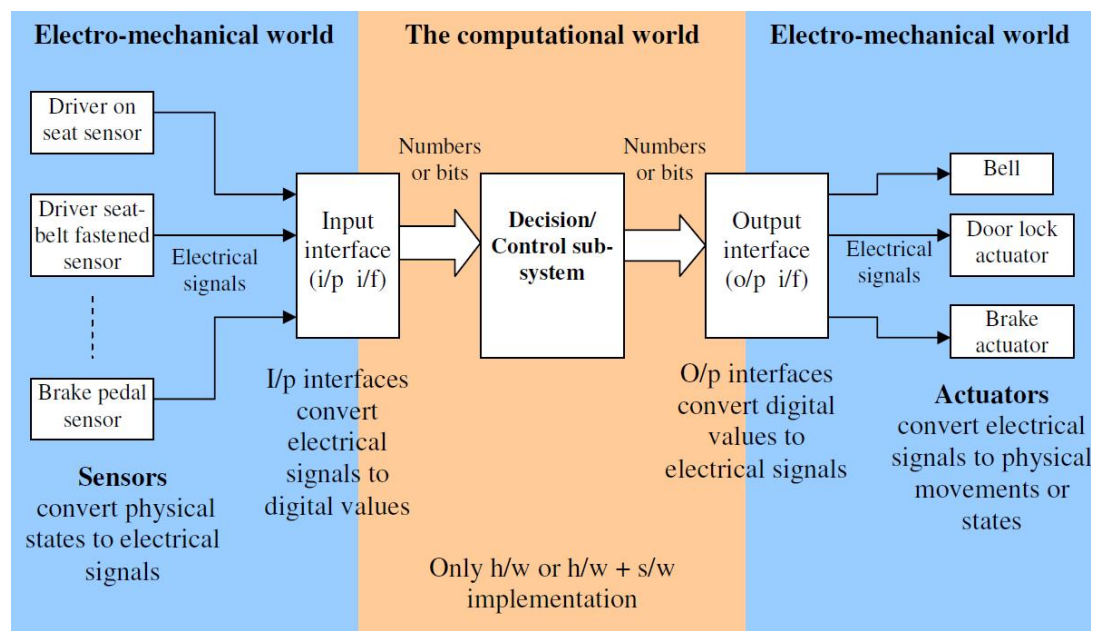
This lab involves in the I/O of chapter 10. Assume you are hired by Ford Motor Company to develop the embedded software for their new model of cars. This software executes on the main computer that sits inside the car. Your manager wrote down the “Software Requirement Specification (SRS)” document and gave it to you. Based on the “requirements” listed in this document you have to develop a C program that materializes these requirements/specifications.

This is a team project (up to three students) which will be completed in two weeks.

2. Lab Instructions or Steps

Given below is a portion of the SRS document which includes the schematic of the car’s electronics and computer systems.

Software Requirement Specification (SRS) Document



The sensors convert the physical states (pressure induced by the driver on the seat when he is seating on it, etc.) and mechanical motions (driver pressing the brake pedal) to electrical signals (voltage, current). The input interface sub-system (commonly denoted as “i/p i/f”, “i/p” for input, “i/f” for interface) converts electrical signal into digital form (number, bits etc.). The decision/control sub-system is the main functional block which provides the intelligence to the car so that it can take the right action based on the given situation. The decision logic provides digital output (numbers, bits), which is converted back to electrical signals by the Output interface (o/p i/f). These electrical signals are used to activate the electro-mechanical actuators (motors, electro-magnets etc.). For this particular lab class, your job is to **design the decision control logic**. You will design the i/p i/f and o/p i/f sub-systems.

Available sensors

Each hardware sensor provides a “high” (1) or “low” (0) output. The input interface sub-system sets the value of the corresponding global integer variable to the respective value. The decision/control logic sub-system will read this integer to take the right decision. It is assumed that this integer is available for C programming. The i/p i/f sub-system keeps monitoring the sensor hardware outputs and takes action (changes value of the integer variable) as soon as the sensor output state changes.

1. DOS – driver on seat. This sensor indicates whether a driver is present. This sensor provides logical “high” (1) as output when a person is sitting on the driver’s seat and “low” (0) if he is not on the seat.
2. DSBF – driver seat belt fastened. This sensor indicates whether the driver seat belt is fastened or not. The sensor hardware provides “high” when driver’s seat belt is fastened, “low” otherwise. The corresponding integer variable that reflects the physical states inside the computer and programming world is “driver_seat_belt_fastened”. The i/p i/f sub-system sets this integer to 1 when DSBF output is high and to 0 when DSBF is low, and the decision/control sub-system code reads/uses it.
3. ER – engine running. This sensor indicates whether engine is running or not. It provides “true” when engine is running, false otherwise. The corresponding integer variable to read and use is “engine_running”.
4. DC – doors closed. Indicates whether all doors are closed or not. The corresponding integer is “doors_closed”.
5. KIC – key in car. Indicates that the keys are still inside the key hole, the corresponding integer variable is “key_in_car”.
6. DLC – door lock lever. This indicates whether the door lock lever is closed or not. To close the electronic door locks the driver has to close this door lock lever. When the car’s computer finds that door lock lever is closed it checks all other variables to assess the situation and finally decides whether to activate the electronic door locks to lock the doors or not. For example if the car keys are still inside but the driver is not on seat (has gone out of car) then the doors should never be locked even though the driver has closed the door lock lever.
7. BP – brake pedal. This indicates that the brake pedal is pressed by the driver.

8. CM – car moving. This sensor indicates the car is moving and at least one of its wheels is turning. The corresponding integer variable “car_moving” has value 1 when the car is moving, and has value 0 if the car is not moving.

Available actuators

1. BELL - A beeper/chime that sounds/plays to alert the driver of any abnormal/hazardous situation. A global integer variable named “bell” is provided in the computer, if your code that implements the decision/control sub-system sets this variable to 1, then the output interface sub-system will read this value and turn the on the voltage on the electrical wire that feeds the beeper/chime. As a result the beeper/chime will start beeping. The beeper will stop when the decision/control subsystem code sets the value of “bell” to 0, because then the o/p i/f will turn down the electrical voltage feeding the beeper hence it will stop. The o/p i/f keeps on monitoring the integer variable and takes action (change the voltage) when the integer changes its value.
2. DLA – door lock actuator. This actuator locks the doors. A corresponding global integer variable named “door_lock” is provided in the computer, if you set this integer to 1 all the doors are locked, it unlocks all doors when you set the value of “door_lock” to 0.
3. BA – brake actuator. This actuator will actually activate the disk brakes in each of the four wheels if the global integer variable “brake” is set to 1. The brake will be released when this variable is set to 0 by the code that implements the decision/control logic sub-system.

Requirements

1. The BELL should chime/sound when the driver starts the engine without fastening his seatbelt.
2. The BELL should sound when the driver starts the car without closing all the doors.
3. The BELL should be off as soon as the conditions change to normal.
4. The doors should not lock when the driver has got out of the car but the keys are still inside the engine, even though the driver has closed the door lock lever. Note: If the driver is on the seat and requests the doors to be locked, the doors must lock.
5. The brake should be engaged when the driver presses the brake pedal. Brakes should disengage when the brake pedal is released. The brake should engage only when the car is moving, when the car is stationary the brake should not unnecessarily engage to reduce mechanical wear and tear of the brake’s hydraulic system.

(End of SRS document section)

Activity 1

For each requirement, separately provide the Boolean expressions that you decided to use.

Activity 2

Create a single combined truth table with all the available sensor inputs and actuator outputs for all the five requirements together. Some of the truth table entries will be don’t-care states

the BELL should not be activated when condition is normal, that is, when both conditions are false. We will provide the Boolean expressions for each of two sub-conditions separately for requirement 1 and 2, and a combined condition that is actually applied to the BELL actuator for requirement 3.

Requirement 1 (seatbelt-not-fastened alert): **!DSBF && ER**.

Requirement 2 (doors-not-closed alert): **!DC && ER**.

Requirement 3 (combined expression for BELL): **(!DSBF || !DC) && ER**. This is actually the disjunction of requirement 1 and 2.

Requirement 4 and 5 specifies the conditions for actuator DLA and BA individually.

Requirement 4 (expression for DLA): **DLC && (DOS || !KIC)**. The expression is a conjunction of two parts. DLC, as the trigger of DLA, is one of them, and DOS || !KIC is the negation of !DOS && KIC, as the only condition (key left in engine when driver is not on seat) at which the door must not be locked.

Requirement 5 (expression for BA): **BP && CM**. The requirement specifies that the brake should activate if brake pedal is pressed and car is moving, and should not be activated if brake pedal is released or when car is stationary. The result is therefore a conjunction of the two.

Activity 2

The truth table is as following. “X” represents irrelevant states (both values of which will lead to the outcome), and “?” represents uncertain outcomes due to the irrelevant states.

DOS	DSBF	ER	DC	KIC	DLC	BP	CM	BELL	DLA	BA
X	0	1	X	X	X	X	X	1	?	?
X	X	1	0	X	X	X	X	1	?	?
X	1	1	1	X	X	X	X	0	?	?
X	X	0	X	X	X	X	X	0	?	?
1	X	X	X	X	1	X	X	?	1	?
X	X	X	X	0	1	X	X	?	1	?
0	X	X	X	1	1	X	X	?	0	?
X	X	X	X	X	0	X	X	?	0	?
X	X	X	X	X	X	1	1	?	?	1
X	X	X	X	X	X	0	X	?	?	0
X	X	X	X	X	X	X	0	?	?	0

Figure 2 Truth table of sensors and actuators

Activity 3

The program code “lab5.c” is given in the appendix section. We designed a few test cases in the input file and ran the program to display outputs. All the outputs match the expected results in the truth table, so it is sufficient to say that the program works as the requirements specifies.

```

ctf-2024101329@ubuntu-lab:~/Documents/computer-organization-labs/lab-3$ cat input.txt
1 0 1 0 1 0 0 0
0 0 1 0 0 0 0 0
1 1 0 1 0 0 0 0
0 0 0 0 1 1 0 0
0 0 0 0 0 1 0 0
1 0 0 0 0 1 0 0
0 0 0 0 0 0 1 1
1 1 1 1 1 1 1 0
ctf-2024101329@ubuntu-lab:~/Documents/computer-organization-labs/lab-3$ ./lab5
^C
ctf-2024101329@ubuntu-lab:~/Documents/computer-organization-labs/lab-3$ cat output.txt
1 0 0
1 0 0
0 0 0
0 0 0
0 1 0
0 1 0
0 0 1
0 1 0
ctf-2024101329@ubuntu-lab:~/Documents/computer-organization-labs/lab-3$

```

Figure 3 Input file with test cases, and output file content after running the program

5. Appendix (Program Code)

Code of lab5.c:

```

/*

This program sounds the bell when driver is on seat AND haven't closed the
doors. Use the general framework and the function shells, replace the code
inside the control_action() function with your own code.

Note: This code is designed to run in an infinite loop to mimic a real control
system. If you prefer to read the inputs from a file, modify the code
appropriately to terminate the loop when all the inputs have been processed.

run this file as : gcc filename.c -o executableName

*/

#include <stdio.h> // This is useful to do i/o to test the code

// sensors
unsigned int driver_on_seat;
unsigned int driver_seat_belt_fastened;
unsigned int engine_running;
unsigned int doors_closed;
unsigned int key_in_car;
unsigned int door_lock_lever;
unsigned int brake_pedal;
unsigned int car_moving;

// actuators
unsigned int bell;
unsigned int door_lock_actuator;
unsigned int brake_actuator;

// operators to control file
FILE *input = NULL;

```

```

FILE *output = NULL;
int end = 0;

void read_inputs_from_ip_if() {
    // place your input code here
    // to read the current state of the available sensors

    // Check end sentinel
    if (end) return;
    // Open input file for the first time
    if (!input) input = fopen("./input.txt", "r");
    // If EOF is reached, close the file and mark isEOF permanently
    if (fscanf(input, "%u", &driver_on_seat) == EOF){
        end = 1;
        fclose(input);
        return;
    }
    // Read the values of the sensors from the file one by one
    fscanf(input, "%u", &driver_seat_belt_fastened);
    fscanf(input, "%u", &engine_running);
    fscanf(input, "%u", &doors_closed);
    fscanf(input, "%u", &key_in_car);
    fscanf(input, "%u", &door_lock_lever);
    fscanf(input, "%u", &brake_pedal);
    fscanf(input, "%u", &car_moving);
}

void write_output_to_op_if() {
    // place your output code here
    // to display/print the state of the 3 actuators (DLA/BELL/BA)

    // Check end sentinel
    if (end) return;
    // Open output file for the first time
    if (!output) output = fopen("./output.txt", "w");
    // Add the values of the actuators to the file in a line
    fprintf(output, "%u %u %u\n", bell, door_lock_actuator, brake_actuator);
    fflush(output);
}

// The code segment which implements the decision logic
void control_action() {
    /*
    The code given here sounds the bell when driver is on seat
    AND hasn't closed the doors. (Requirement-2)
    Replace this code segment with your own code.
    */

    bell = engine_running && (!driver_seat_belt_fastened || !doors_closed);
    door_lock_actuator = door_lock_lever && (driver_on_seat || !key_in_car);
    brake_actuator = brake_pedal && car_moving;
}

/* --- You should not have to modify anything below this line -----*/

int main(int argc, char *argv[]) {
    /*The main control loop which keeps the system alive and responsive for
    ever, until the system is powered off */

```

```
    for (;;) {  
        read_inputs_from_ip_if();  
        control_action();  
        write_output_to_op_if();  
    }  
  
    return 0;  
}
```